

ARFL

Privacy in the Dark Cloud

A Decentralised Privacy Protocol Powered by Bitcoin

arfl.io · github.com/arfl-protocol · #arfl on Nostr

Abstract. *The VPN industry sold privacy. What it actually sold was trust in a company — a company with servers that can be seized, a CEO that can be pressured, and invoices that link identity to browsing history. ARFL is a decentralised privacy protocol that eliminates all three problems. ARFL aggregates four open protocols — WireGuard for encrypted transport, Nostr for pseudonymous node discovery, the Bitcoin Lightning Network for per-bundle micropayments, and Fedimint for trustless deposit custody — into a self-sustaining privacy network that anyone can build on. Node operators plug directly into the ARFL protocol, earning Bitcoin passively from bandwidth they already own. Hubs are products built on top of the protocol, competing on brand and service rather than infrastructure. A user's WireGuard keypair — generated locally and never transmitted — is their sole identity. Nobody owns the network. Nobody controls the network. The protocol defines the rules. Mathematics enforces them.*

Draft v0.1 · 2026 · No rights reserved

1. Introduction

Commerce on the internet has come to rely almost entirely on centralised intermediaries to provide privacy services. The VPN industry, despite its promises, reproduces the same structural problem: a central company holds the keys, the logs, and the liability. That company can be subpoenaed, hacked, acquired, or pressured. Its users have no recourse and no verification. They are asked to trust. Trust is not privacy.

The subscription model compounds the problem. Monthly payments require credit cards. Credit cards require identity. Users sign up at promotional rates and renew at multiples of the original price — surprised, locked in, and no more private than before. Every centralised VPN, regardless of its privacy claims, can be subpoenaed, hacked, acquired, or pressured into policy changes. The architecture is the vulnerability.

What is needed is a privacy network based on cryptographic proof and economic incentive rather than trust — one where no central server exists to seize, no account exists to subpoena, and no company exists to pressure. In this paper we propose ARFL: a decentralised privacy protocol that aggregates four existing open protocols into a self-sustaining privacy network owned by nobody and available to everyone.

ARFL defines two classes of participant. Node operators interact directly with the protocol, contributing bandwidth and earning Bitcoin. Hubs are products built on top of the protocol, providing user-facing experiences and competing on brand and service quality. The protocol itself owns no infrastructure, controls no funds, and requires no permission to build on.

The key properties of ARFL are: (1) no user accounts or identity — a locally generated WireGuard keypair is the user’s sole credential; (2) no central server — node operators publish availability pseudonymously via Nostr; (3) no subscription — users purchase bandwidth bundles via Bitcoin Lightning; (4) no logs — the architecture makes logging impossible, provable by inspection of open source code; (5) no token — the incentive mechanism is Bitcoin.

2. System Architecture

ARFL is composed of four protocol layers and two participant layers. No new cryptography is introduced. The contribution of ARFL is the composition of existing protocols into a coherent privacy system with aligned economic incentives.

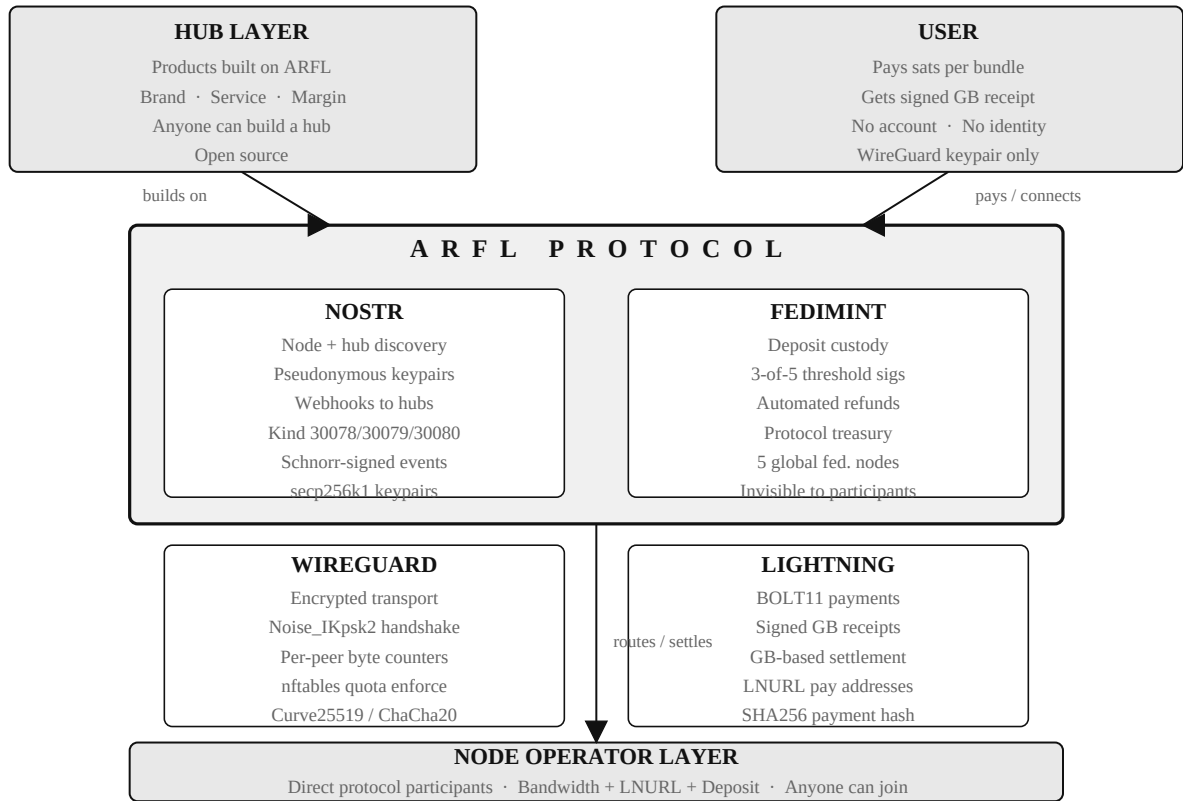


Figure 1. ARFL protocol architecture.

The session data flow proceeds through a mandatory two-hop routing design. No single node holds the complete picture of a user session:

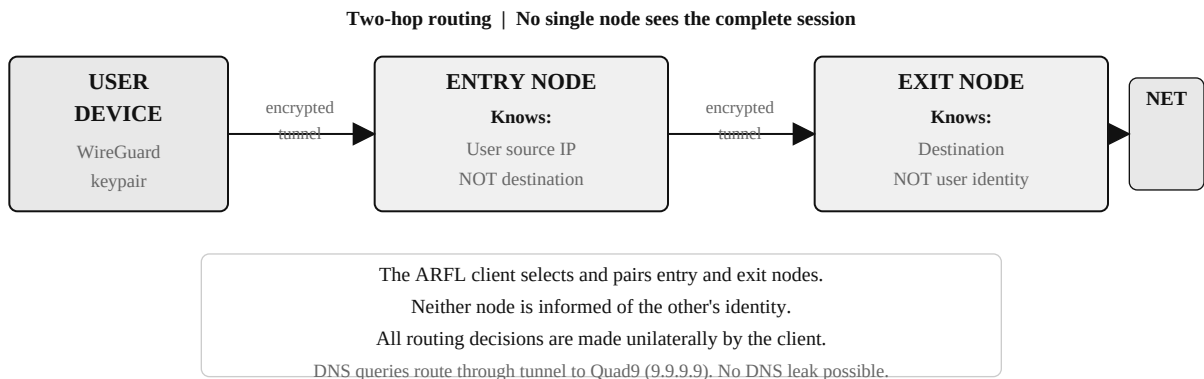


Figure 2. Two-hop session routing.

3. Cryptographic Primitives

ARFL inherits its cryptographic security from four independent protocol stacks, each well-audited and widely deployed in production systems.

3.1 WireGuard

WireGuard implements the Noise_IKpsk2 handshake protocol, providing mutual authentication and forward secrecy. The cryptographic primitives are fixed and non-negotiable, eliminating downgrade attacks by design:

- **Curve25519** — Elliptic-curve Diffie-Hellman key exchange.
- **ChaCha20-Poly1305** — Authenticated encryption (AEAD) for all tunnel traffic.
- **BLAKE2s** — Cryptographic hashing for key derivation and MAC operations.
- **SipHash24** — Hashtable keying to prevent denial-of-service via hash collision.

Each WireGuard peer is identified by a 256-bit Curve25519 public key. The ARFL client generates a keypair locally on first launch and stores the private key in the device secure enclave. This keypair is the user's sole persistent identity — no username, no email, no password.

3.2 Nostr

Nostr uses secp256k1 elliptic curve keypairs for identity. Every node and hub announcement is a Nostr event signed with a Schnorr signature over the SHA256 hash of the canonical event serialisation (NIP-01). Operators are identified solely by their Nostr public key — no real-world identity is required or stored.

```
Event ID: SHA256(canonical_json_serialisation)
Signature: Schnorr(private_key, event_id)
Encoding: NIP-19 bech32 (npub / nsec) for display
Transport: WebSocket to Nostr relays
```

3.3 Lightning Network

Payments are made via BOLT11 invoices. Each invoice encodes a SHA256 payment hash derived from a secret preimage known only to the payee. Payment is cryptographically proven by revealing the preimage. The entire ARFL economy is denominated in satoshis. USD figures are human-readable references only and carry no protocol obligation.

3.4 Fedimint

Fedimint provides federated custody of node operator security deposits using threshold signatures. The ARFL protocol team operates a Fedimint federation of five nodes across separate global jurisdictions, requiring three-of-five signatures to release or slash any deposit. This eliminates single-operator control of node operator funds while remaining completely

transparent to participants — the deposit address is indistinguishable from any Lightning address.

4. Node Discovery via Nostr

Node operators announce availability by publishing signed Nostr events to public relays. The ARFL client subscribes to these relays and maintains a live index of available nodes, updated continuously as new events arrive. No central directory exists. No registration is required.

4.1 Node Announcement Event

ARFL defines a custom Nostr event kind 30078 in the parameterised replaceable event range (30000–39999 per NIP-01), allowing relays to replace stale announcements by the same pubkey and ‘d’ tag. Node software republishes this event every 60 seconds:

```
{
  "kind": 30078,
  "pubkey": "<node_nostr_pubkey_hex>",
  "created_at": <unix_timestamp>,
  "tags": [
    ["d", "<unique_node_id>"],
    ["wg_pubkey", "<wireguard_pubkey_base64>"],
    ["endpoint", "<ip_address>:<port>"],
    ["lnurl", "<lnurl_pay_address>"],
    ["deposit", "<deposit_sats>"],
    ["upload", "<mbps>"],
    ["download", "<mbps>"],
    ["load", "<current_user_count>"],
    ["capacity", "<max_user_count>"],
    ["role", "both|entry|exit"],
    ["version", "<arfl_node_version>"]
  ],
  "content": "",
  "id": "<sha256_of_canonical_serialisation>",
  "sig": "<schnorr_signature>"
}
```

4.2 Hub Announcement Event

Hubs publish their own Nostr announcement using event kind 30079. The client reads all available hubs and selects automatically based on reputation and uptime, or allows manual selection by the user:

```
{
  "kind": 30079,
  "pubkey": "<hub_nostr_pubkey_hex>",
```

```

"tags": [
  ["d", "<unique_hub_id>"],
  ["name", "<hub_name>"],
  ["lnurl", "<hub_lightning_address>"],
  ["price_sats", "<bundle_price_in_sats>"],
  ["uptime", "<percentage>"],
  ["version", "<arfl_hub_version>"]
],
"content": "",
"id": "<sha256_of_canonical_serialisation>",
"sig": "<schnorr_signature>"
}

```

4.3 Node Selection Algorithm

The client selects an entry and exit node pair on each new session. Critically, neither node is informed of the other's identity — the pairing exists only within the client for the duration of the session:

```

1. Filter: status=online, load < capacity,
deposit >= minimum, version compatible
2. Score each node:
score = (capacity - load) / capacity
* upload_mbps / 50
* uptime_ratio
3. Select entry node: highest score, role=entry|both
4. Select exit node: highest score, role=exit|both,
different operator pubkey to entry
5. Pairing established client-side only.
Neither node is informed of the other.

```

4.4 Collusion Prevention

The two-node design prevents collusion by construction. Entry and exit nodes are selected independently and identified only by pseudonymous Nostr keypairs. They never communicate directly. All routing decisions are made unilaterally by the client. An operator running multiple nodes under different keypairs cannot coordinate a fraudulent session because the nodes have no awareness of each other's involvement.

5. VPN Transport via WireGuard

WireGuard provides the encrypted transport layer. ARFL node software manages WireGuard programmatically using the wgctrl library (golang.zx2c4.com/wireguard/wgctrl, MIT licensed), which communicates with the WireGuard kernel module via generic netlink. The CLI is never used in production.

5.1 Session Establishment

A complete ARFL session proceeds as follows. The session token in step 2 is described in full in Section 10.3.

```
1. Client reads Nostr, selects entry + exit node pair.
2. Client generates session token:
   token = BLAKE2s(wg_pubkey || timestamp || nonce)
3. Client presents to hub: (wg_pubkey, session_token)
4. Hub verifies signed GB receipt: balance > 0, not expired.
5. Hub returns: (entry_endpoint, exit_endpoint, session_id)
6. Hub adds client peer on entry node via wgctrl:
   ConfigureDevice('arfl0', PeerConfig{
     PublicKey: client_wg_pubkey,
     AllowedIPs: [client_tunnel_ip/32],
   })
7. Hub adds entry node as allowed peer on exit node:
   ConfigureDevice('arfl0', PeerConfig{
     PublicKey: entry_wg_pubkey,
     AllowedIPs: [entry_tunnel_ip/32],
   })
8. Client initiates WireGuard handshake to entry node.
9. Noise_IKpsk2 handshake completes -- tunnel established.
10. Traffic flows: client -> entry -> exit -> internet.
11. DNS set to 9.9.9.9 (Quad9) within tunnel -- no DNS leak.
```

5.2 Per-Peer Byte Accounting

WireGuard maintains monotonic 64-bit byte counters per peer (ReceiveBytes and TransmitBytes), updated per-packet in the kernel data path. The ARFL node daemon polls these counters every five seconds via wgctrl. Counters persist across reconnects and rekeying, resetting only on peer removal. The daemon persists the last known counter value to disk to survive process restarts:

```
func pollBytes(iface string, pubkey wgtypes.Key) {
    prev := loadFromDisk(pubkey)
    for {
        dev, _ := wgClient.Device(iface)
        for _, peer := range dev.Peers {
            if peer.PublicKey == pubkey {
                current := peer.ReceiveBytes + peer.TransmitBytes
                delta := current - prev
                reportToHub(pubkey, delta)
                saveToDisk(pubkey, current)
                prev = current
            }
        }
    }
}
```

```

time.Sleep(5 * time.Second)
}
}

```

5.3 Quota Enforcement

Quota is enforced in 256 MB kernel slabs via nftables. As each slab is consumed the hub refreshes the quota element. When the GB balance reaches zero the hub does not refresh and removes the peer. This two-layer approach eliminates the polling overshoot window: the kernel enforces the hard stop independently of the polling cycle.

```

# Apply quota slab at session start
nft add element inet arfl quotas {
    &lt;client_tunnel_ip> : quota over 268435456 bytes
}

# On balance exhausted -- remove peer
wgClient.ConfigureDevice('arfl0', wgtypes.Config{
    Peers: []wgtypes.PeerConfig{{
        PublicKey: clientPubKey,
        Remove: true,
    }},
})

```

5.4 Key Management and Device Portability

The user's WireGuard private key is generated locally on first launch and stored in the device secure enclave. It is never transmitted to any ARFL component. The public key is the user's sole network identity, linked to their GB receipt.

To migrate to a new device, the user exports their private key as a QR code from the original device and imports it on the new device. The GB receipt follows the key. If the key is lost, the balance is irrecoverable by design. This is an intentional tradeoff: the same property that prevents account recovery prevents identity linkage.

6. Payment Architecture

ARFL introduces the first pay-per-use VPN payment model. Users purchase bandwidth in discrete bundles. No subscription, no account, no identity. The hub acts as the payment gateway — receiving from users and settling to node operators.

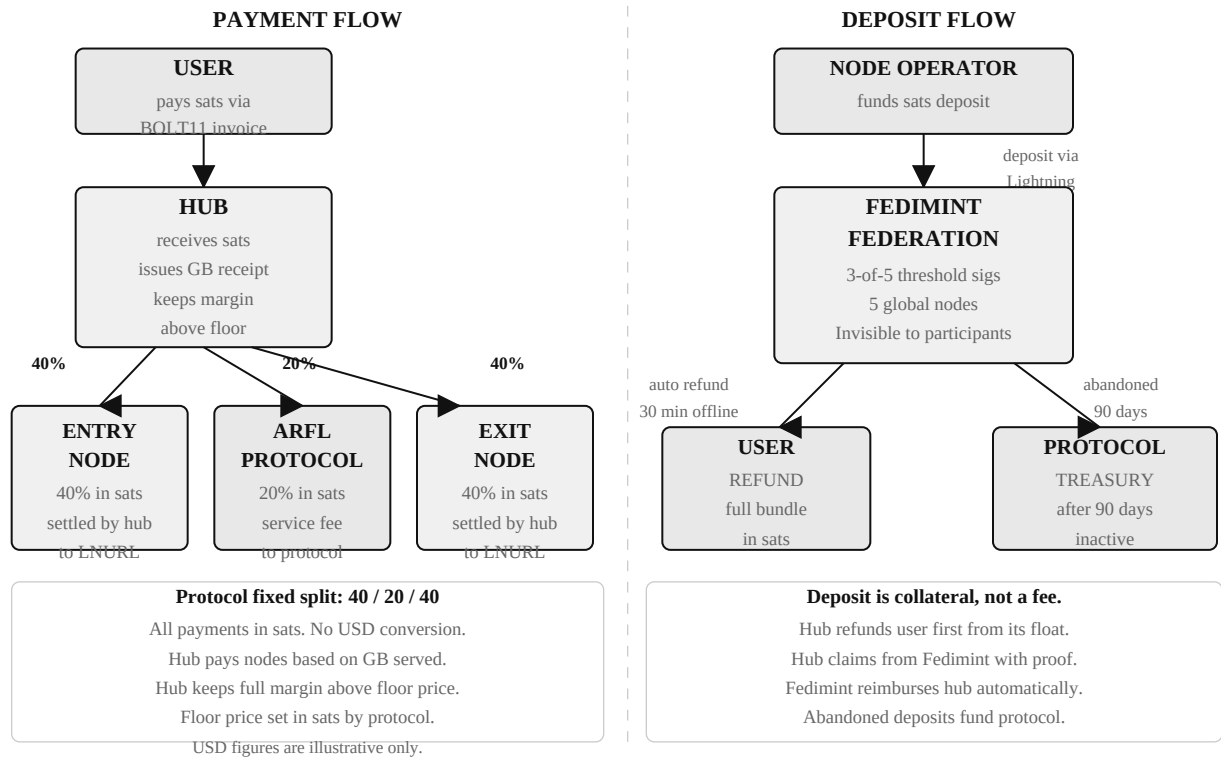


Figure 3. Payment and deposit flow.

6.1 Protocol Fixed Values

The following values are fixed at the protocol level and enforced by compliant client software. They can only be changed by community consensus and a protocol upgrade:

Parameter	Value	Rationale
Bundle size	250 GB	Standard unit of bandwidth
Floor price	Fixed in sats	Protects node and hub margins
Node deposit	Fixed in sats	Refund collateral per node
Revenue split	40 / 40 / 20	Entry / exit / hub
Bundle expiry	30 days	Encourages active use
Min bandwidth	50 Mbps up/down	Baseline service quality
Ping interval	60 seconds	Node health monitoring
Offline trigger	30 minutes	Refund activation threshold
DNS resolver	9.9.9.9 Quad9	Privacy-preserving resolver
Settlement	Per GB served	Based on actual usage

Table 1. ARFL protocol fixed values.

6.2 Purchase Flow

The complete purchase flow from user payment to GB credit. All amounts are denominated in satoshis at the live market rate at the moment of purchase:

```
1. User selects hub from Nostr discovery.
2. App prompts for Lightning refund address (optional LNURL).
   Users without one waive automated refund rights.
3. App sends to hub: (wg_pubkey, refund_lnurl)
4. Hub generates BOLT11 invoice:
   amount = floor_price_sats at live rate
   description = 'ARFL 250GB bundle'
   expiry = 3600 seconds
5. User pays invoice from any Lightning wallet.
6. Hub receives payment preimage -- confirms settlement.
7. Hub issues signed GB receipt to client:
{
  wg_pubkey: <curve25519_pubkey>;,
  gb_total: 250,
  gb_remaining: 250,
  expiry: <unix_timestamp>;,
  refund_lnurl: <user_address>;,
  hub_sig: <schnorr_signature>;
}
8. Receipt stored on client device. Not on hub.
9. App displays: '250 GB available. Expires <date>;.'
```

Active sessions are unaffected by hub downtime.
WireGuard tunnels remain open independent of hub availability.
New purchases are paused until the hub returns.

6.3 Revenue Split and Daily Settlement

The hub accumulates node operator earnings and settles daily via Lightning to the LNURL published in each node's Nostr announcement. The split applies to the floor price. Hub margin above the floor is retained entirely:

```
Per 250 GB bundle at floor price:
Entry node: 40% --> node LNURL via Lightning
Exit node: 40% --> node LNURL via Lightning
Hub: 20% --> hub retained

Hub charges above floor:
Entry node: 40% of floor in sats --> node LNURL
Exit node: 40% of floor in sats --> node LNURL
Hub: 20% of floor in sats + full margin above floor
```

```
Settlement:
Hub tracks GB served per node via WireGuard byte counters.
Hub pays nodes in sats based on GB served.
Node LNURL sourced from latest signed Nostr announcement.
All amounts in sats. USD figures are illustrative only.
```

6.4 Wallet Sovereignty

ARFL places no requirements on the type of Lightning wallet used by users or node operators. Custodial wallets, non-custodial wallets, KYC and KYC-free wallets are all compatible — any wallet capable of paying a BOLT11 invoice and receiving to an LNURL is sufficient. Privacy and custody decisions are the sole responsibility of the wallet holder. ARFL is a protocol, not a custodian.

7. Deposit and Refund Model

Node operators fund a security deposit before joining the network. This deposit is held in trustless custody by the ARFL Fedimint federation and serves as collateral against service failure. The entire deposit and refund lifecycle is automated.

7.1 Fedimint Federation

The ARFL protocol team operates a Fedimint federation of five nodes in separate global jurisdictions. Three-of-five threshold signatures are required to release or slash any deposit. From the node operator's perspective the deposit address is indistinguishable from any Lightning address — the Fedimint architecture is completely transparent to all participants.

In future protocol versions, federation membership will be opened to community operators, progressively decentralising deposit custody consistent with the broader ARFL architecture.

7.2 Automated Refund Flow

The hub monitors every active node via signed ping events every 60 seconds. When a node is offline for 30 consecutive minutes the following automated sequence executes without human intervention:

```
1. Hub records 30 consecutive signed failed ping timestamps.
2. Hub pays full bundle sats to user refund_lnurl immediately.
   Hub float absorbs this. User is refunded at once.
3. Hub submits proof bundle to Fedimint:
{
  node_pubkey: &lt;offline_node&gt;;
  offline_proof: &lt;30 signed failed ping timestamps&gt;;
```

```
refund_proof: <lightning_payment_proof>;  
refund_sats: <full_bundle_amount>;  
hub_lnurl: <reimburse_here>;  
}  
4. Fedimint verifies automatically:  
- Node pubkey matches registered deposit.  
- 30 minutes offline confirmed via signed timestamps.  
- Lightning refund payment confirmed.  
- Amount <= remaining deposit balance.  
- No duplicate claim for this session.  
5. Fedimint releases sats to hub LNURL. Hub is made whole.  
6. Node deposit balance reduced by full bundle amount.  
7. If deposit hits zero: node auto-delisted from Nostr routing.
```

The ARFL protocol guarantees full bundle refunds from the node operator deposit. Reliability is binary: either the service was provided or it was not. No partial refunds are issued. Refunds are denominated and paid in satoshis at the time of refund. The protocol makes no guarantee of USD equivalence. Users accept Bitcoin price volatility as a condition of participation — this is consistent with paying in Bitcoin in the first place. Hubs charging above the floor price are responsible for refunding their margin from their own reserves.

7.3 Deposit Top-Up

When a node deposit falls below the minimum threshold, the node software automatically alerts the operator and generates a Lightning invoice for top-up. The operator pays from their earnings wallet. The federation credits the deposit and the node is immediately relisted on Nostr. This is the only manual action in the entire refund lifecycle.

7.4 Protocol Treasury

Node operators who abandon the network forfeit their remaining deposit. After 90 days of inactivity the federation automatically transfers the remaining balance to the ARFL protocol treasury. These funds support protocol maintenance and development. The deposit is a participation commitment, not a reversible payment. At 10 USD and given that operators typically recoup this within days of joining, forfeiture on abandonment is an acceptable and intentional tradeoff.

8. Webhook Architecture

The ARFL protocol publishes real-time events to subscribed hubs via encrypted Nostr direct messages (NIP-04). Hubs subscribe by publishing a kind 30080 Nostr event specifying their webhook endpoint and pubkey. This enables hubs to monitor network state, automate responses, and build differentiated user experiences on top of a shared protocol.

8.1 Event Types

Event	Trigger	Hub Action
NODE_OFFLINE	Node fails ping	Start 30-min timer
NODE_ONLINE	Node resumes ping	Cancel timer if active
REFUND_TRIGGERED	30-min offline confirmed	Pay user, claim Fedimint
DEPOSIT_LOW	Deposit below threshold	Alert node operator
DEPOSIT_DEPLETED	Deposit reaches zero	Delist node from routing
GB_BALANCE_LOW	Receipt below 25 GB	Push purchase prompt
SESSION_STARTED	WireGuard tunnel established	Log session start
SESSION_ENDED	Tunnel closed	Update GB accounting
SETTLEMENT_SENT	Daily payment to node	Reconcile ledger

Table 2. ARFL webhook event types and recommended hub actions.

8.2 Hub Subscription Event

```
{
  "kind": 30080,
  "pubkey": "<hub_nostr_pubkey>",
  "tags": [
    ["d", "<hub_id>"],
    ["webhook", "https://hub.example.com/arfl/events"],
    ["events", "NODE_OFFLINE, REFUND_TRIGGERED,
      GB_BALANCE_LOW, DEPOSIT_LOW"]
  ],
  "content": "",
  "sig": "<schnorr_signature>"
}
```

Protocol events are delivered encrypted to the hub's Nostr pubkey via NIP-04 direct messages. Hubs may additionally specify an HTTPS webhook endpoint for direct POST delivery. Both delivery mechanisms are supported for redundancy.

9. Privacy Architecture

ARFL is designed so that no single entity accumulates sufficient information to reconstruct a user session:

Component	Knows	Does Not Know
Entry node	User source IP	Destination, user identity
Exit node	Traffic destination	User IP, user identity
Hub	WireGuard pubkey, receipt sig	Browsing, node pairing, IP
Nostr relay	Node and hub announcements	User activity, sessions
Fedimint	Deposit balances	Sessions, user identity
Anyone	Nothing complete	Full session picture

Table 3. Information compartmentalisation across ARFL components.

DNS queries are routed through the encrypted WireGuard tunnel to Quad9 (9.9.9.9) — a privacy-preserving resolver with a published no-logs policy. This is enforced in the WireGuard interface configuration delivered at session start. The exit node never sees DNS queries. No DNS leak is architecturally possible.

The hub stores only the WireGuard public key, receipt signature, expiry timestamp, and refund LNURL — none of which constitute identity. Because the hub is open source, this claim is verifiable by inspection, not merely asserted in a privacy policy.

10. Security Model

10.1 Threat Model

ARFL protects against:

- ISP traffic surveillance and deep packet inspection.
- Network-level IP address tracking and logging.
- DNS query surveillance and domain-level logging.
- Identity-linked browsing history.
- Legal compulsion of a central VPN provider.
- Server seizure by authorities.
- Corporate acquisition or policy change.

ARFL does not protect against:

- Malware or compromise of the user’s own device.
- Browser fingerprinting techniques.
- Services the user voluntarily authenticates into.
- A coordinated adversary controlling a majority of the node network simultaneously.

10.2 Sybil Resistance

An attacker attempting to control a large fraction of the node network must fund a 10 USD deposit per node identity, all held at risk of depletion through refund obligations. At 1,000 nodes this requires 10,000 USD in capital. The deposit requirement raises the cost of a network-wide deanonymisation attack to a level disproportionate to the value of information gained.

10.3 Replay Attack Prevention

Each session request includes a token derived from the user's WireGuard public key, the current timestamp, and a random nonce: $\text{token} = \text{BLAKE2s}(\text{wg_pubkey} \parallel \text{timestamp} \parallel \text{nonce})$. The hub maintains a short-lived token cache and rejects duplicate tokens within a five-minute window, preventing replay of valid session requests.

10.4 WireGuard Traffic Analysis Resistance

WireGuard is silent by design: it does not respond to unauthenticated packets and sends no response to packets that fail authentication. This makes ARFL nodes invisible to port scanners and resistant to active probing. The fixed-suite cryptography eliminates cipher negotiation as an attack surface.

11. Trust Model

ARFL is designed to require no trust in any single party. Each security property is enforced by cryptographic guarantees, economic incentives, or open source verifiability:

Property	Enforcement
Encrypted transport	WireGuard ChaCha20-Poly1305 -- cryptographic
No user identity	WireGuard keypair -- locally generated, never transmitted
No session logs	Architecture -- no component sees full session
Node availability	Security deposit -- economic penalty for downtime
Payment proof	BOLT11 preimage -- cryptographic proof of payment
Node authenticity	Nostr Schnorr signature -- cryptographic
Quota enforcement	nftables kernel quota -- OS enforced
Deposit custody	Fedimint threshold signatures -- 3 of 5 required
No DNS leak	WireGuard DNS config -- architectural

Hub compliance	Client validation + open source + community audit
----------------	---

Table 4. Trust properties and enforcement mechanisms.

The hub requires bounded trust: it cannot access browsing history, cannot deanonymise users, and cannot steal funds beyond the brief window between payment and receipt issuance. Its source code is open and auditable. The Fedimint federation requires trust in the protocol team in version 1, with progressive decentralisation to community operators in subsequent versions.

12. Node Operator Economics

ARFL node operators earn Bitcoin passively by contributing bandwidth to the network. The economic model is analogous to Bitcoin proof-of-work: operators contribute a resource, are rewarded proportionally to contribution, and earn nothing when offline. The Airbnb parallel is also instructive: node operators list their bandwidth as hosts list their properties — they do not receive free VPN access any more than an Airbnb host receives free stays elsewhere. Users and node operators are entirely separate participant classes.

The initial cost to participate as both node operator and user is modest: a 10 USD deposit to join the network and a 5 USD bundle to use the VPN, totalling 15 USD. At modest traffic levels of ten simultaneous users, a node earns approximately 48 USD per month, recouping the full participation cost in under ten days.

Node Activity	Simultaneous Users	GB Served / Month	Est. Monthly Earnings
Quiet	~10	~2 TB	~48 USD in sats
Active	~50	~10 TB	~240 USD in sats
Busy	~100	~20 TB	~480 USD in sats

Table 5. Estimated node operator monthly earnings by activity level.

Nodes with faster connections serve more simultaneous users and earn proportionally more at the same per-GB rate. The market self-selects for quality without tiered pricing or central intervention. The minimum hardware requirements are deliberately low:

- 50 Mbps upload and download minimum.
- Any Lightning wallet supporting LNURL — no full Lightning node required.
- 10 USD security deposit in satoshis.
- 80% uptime commitment.
- ARFL node software (open source, Go).

13. Hub Economics

Hubs earn 20% of the floor price on every bundle sold, plus the full margin on any amount charged above the floor. Hub infrastructure costs are fixed and low — a well-connected Lightning node and a modest server — making the hub economically viable at small user volumes and highly profitable at scale.

Hubs compete on brand, reliability, and service quality rather than on price. Since all compliant hubs share the same node network, network quality is equal infrastructure. Differentiation comes from the user experience built on top of the protocol: how quickly a hub responds to node failures, how well it notifies users, what premium features it offers, and the trust its brand commands in the community.

The floor price is enforced by the client application. Any hub publishing a price below the floor is rejected as non-compliant. This protects node operator earnings and prevents a race to the bottom that would undermine network quality. Any operator disagreeing with the floor price may fork the open source protocol, but at 5 USD per 250 GB the floor is so competitive that no rational actor benefits from defection.

14. Why No Token

Every prior decentralised privacy network introduced a native token: Mysterium (MYST), Sentinel (DVPN), Orchid (OXT). Tokens introduce speculation over utility, regulatory uncertainty across jurisdictions, participation barriers for users unfamiliar with token acquisition, and protocol complexity that distracts from the underlying mission.

ARFL uses Bitcoin — the monetary network that already exists, already works, and is already held by the community most aligned with ARFL's values. The Lightning Network provides the micropayment infrastructure that makes per-bundle pricing practical at negligible transaction cost. No new token is introduced. No ICO is conducted. No token treasury exists. The incentive mechanism is Bitcoin, and Bitcoin alone.

15. Conclusion

We have proposed a decentralised privacy protocol that requires no trust in any central authority. By aggregating WireGuard for encrypted transport, Nostr for pseudonymous discovery, the Bitcoin Lightning Network for per-bundle payments, and Fedimint for trustless deposit custody, ARFL creates a privacy network where node operators are rewarded for reliability, users pay only for what they use, hubs compete on service rather than infrastructure, and no single point of failure can compromise the network.

The architecture makes two clean separations. Node operators interact directly with the protocol — contributing bandwidth, earning Bitcoin, with no hub relationship required. Hubs build products on top of the protocol — providing user experiences, competing on brand, with no infrastructure to own. The protocol itself owns nothing, controls nothing, and requires no permission to build on.

The user’s WireGuard keypair — generated locally, stored in the device secure enclave, never transmitted — is their sole identity. The network is robust in its simplicity. Node operators require only bandwidth and a Lightning wallet. Users require only satoshis and the ARFL application. The protocol requires only open source code, cryptographic guarantees, and economic incentive.

Privacy is not a product. It is a right. ARFL is the infrastructure for that right.

References

- [1] D.J. Bernstein. “Curve25519: New Diffie-Hellman Speed Records.” PKC 2006.
<https://cr.yp.to/ecdh/curve25519-20060209.pdf>
- [2] Y. Nir, A. Langley. “ChaCha20 and Poly1305 for IETF Protocols.” RFC 8439. 2018.
<https://datatracker.ietf.org/doc/html/rfc8439>
- [3] Z. Donenfeld. “WireGuard: Next Generation Kernel Network Tunnel.” NDSS 2017.
<https://www.wireguard.com/papers/wireguard.pdf>
- [4] WireGuard control library. wgctrl-go. MIT License. <https://github.com/WireGuard/wgctrl-go>
- [5] fiatjaf et al. “Nostr Protocol — NIP-01: Basic protocol flow.”
<https://github.com/nostr-protocol/nostr/blob/master/01.md>
- [6] fiatjaf et al. “Nostr Protocol — NIP-04: Encrypted Direct Messages.”
<https://github.com/nostr-protocol/nostr/blob/master/04.md>
- [7] fiatjaf et al. “Nostr Protocol — NIP-19: bech32-encoded entities.”
<https://github.com/nostr-protocol/nostr/blob/master/19.md>
- [8] J. Poon, T. Dryja. “The Bitcoin Lightning Network.”
<https://lightning.network/lightning-network-paper.pdf>, 2016.
- [9] Lightning Network Specification. BOLT11: Invoice Protocol.
<https://github.com/lightning/bolts/blob/master/11-payment-encoding.md>
- [10] LNURL Specification. <https://github.com/lnurl/luds>
- [11] Fedimint. Federated Chaumian Mints. <https://github.com/fedimint/fedimint>
- [12] T. Perrin. “The Noise Protocol Framework.” <https://noiseprotocol.org/noise.html>, 2018.
- [13] S. Nakamoto. “Bitcoin: A Peer-to-Peer Electronic Cash System.” <https://bitcoin.org/bitcoin.pdf>, 2008.
- [14] Quad9. Privacy-preserving DNS resolver. <https://www.quad9.net>
- [15] Mysterium Network. Decentralised VPN Protocol. <https://github.com/mysteriumnetwork/node>
- [16] Sentinel. dVPN Protocol. <https://github.com/sentinel-official>